

RAG Knowledge Base: ChromaDB PDF Sample

Retrieval augmented generation, usually called RAG, helps an application answer questions by searching a trusted knowledge base before producing a response. Instead of asking a language model to rely only on its training data, the application retrieves relevant context and sends that context with the user's question.

A typical PDF ingestion pipeline has four steps. First, the document loader extracts readable text from each PDF page. Second, the text splitter breaks long page text into smaller overlapping chunks. Third, an embedding model converts every chunk into a numeric vector. Fourth, ChromaDB stores the vectors, the original chunk text, and metadata such as source file, page number, and chunk id.

Chunk size matters because it controls how much context each retrieved result contains. Chunks that are too small may lose important details. Chunks that are too large may mix unrelated topics and reduce retrieval quality. A common starting point is 500 to 1,000 characters with 50 to 150 characters of overlap.

Metadata makes the vector database easier to use. For PDF content, useful metadata includes document name, page number, section title, creation date, and data owner. When the application returns an answer, metadata can be used to show citations so the user can verify where the information came from.

ChromaDB can run locally for development and persist data to a folder on disk. A collection is similar to a table for embeddings. Each collection contains ids, documents, embeddings, and optional metadata. During a query, ChromaDB embeds the search text, compares it with stored vectors, and returns the nearest chunks.

This sample document is intentionally small but realistic. It contains enough information to test PDF loading, text chunking, embedding creation, persistence, and similarity search. A good first query after ingestion is: What metadata should I store for PDF chunks?